Material didático - Estrutura Sequencial

Site: Ambiente Virtual de Aprendizagem - UFR
Course: ALGORITMOS E PROGRAMAÇÃO
Book: Material didático - Estrutura Sequencial

Printed by: Nabor do Nascimento da Silva
Date: Sunday, 5 April 2026, 4:44 PM

Table of contents

1. Apresentação

2. Algoritmos
2.1. Tipos de algoritmos
2.2. Não computacionais
2.3. Computacionais

3. Projeto de algoritmos
3.1. Linguagem natural
3.2. Fluxogramas
3.3. Pseudocódigo
3.4. Para saber mais: comandos em pseudocódigo
3.5. Linguagens de Programação

4. Estrutura fundamental de um algoritmo
4.1. Cabeçalho
4.2. Área de declaração de variáveis
4.3. Programa principal e subprogramas
4.4. Documentação interna
4.5. Indentação

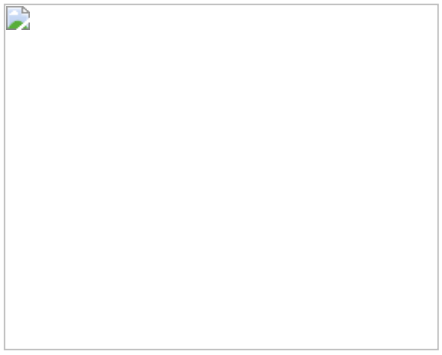
5. Estruturas de controle

6. Estrutura Sequencial
6.1. Organização lógica da estrutura sequencial

7. Categorias de problemas

8. Verificação e validação de algoritmos
8.1. Teste de Mesa
8.2. Exemplo de teste de mesa
8.3. Como elaborar caso de teste

1. Apresentação



Crédito da imagem: Imagem de Freepik

Apresento a seguir os conceitos de algoritmos e projeto de algoritmos.

Bom estudo!
Prof. Waine

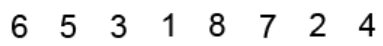
2. Algoritmos

O conceito de algoritmo

Um algoritmo é uma sequência ordenada e lógica de instruções ou passos que definem um conjunto de operações a serem executadas para resolver um problema ou realizar uma tarefa específica.

De forma bem simples, trata-se de um tipo de representação de um processo que foi compreendido. Então ele é descrito em um tipo de representação para que possa ser executado novamente.

Veja a animação a seguir.



A animação acima mostra a execução de um algoritmo. Você consegue descobrir qual o objetivo desse algoritmo? Você consegue escrever quais são os passos que ele executa?

Definições

Em matemática e ciência da computação, um algoritmo é uma sequência finita de ações executáveis que visam obter uma solução para um determinado tipo de problema.

Veja algumas definições encontradas na literatura:

A palavra "**algoritmo**" faz referência ao matemático árabe Al Khwarizmi, que viveu no século IX, e descreveu regras para equações matemáticas.

Na **MATEMÁTICA**: sequência finita de regras, raciocínios ou operações que, aplicada a um número finito de dados, permite solucionar classes semelhantes de problemas.

Na **INFORMÁTICA**: conjunto das regras e procedimentos lógicos perfeitamente definidos que levam à solução de um problema em um número finito de etapas.

Algoritmos são usados para resolver **problemas computacionais**, que podem envolver manipulação de dados, ordenação, pesquisa, processamento de imagens, aprendizado de máquina e muitos outros domínios da ciência da computação.

Importância

Os algoritmos permitem aos programadores criar soluções eficientes e lógicas tanto para problemas simples quanto para problemas complexos.

Algoritmos podem ser escritos em linguagem natural, de forma que podem ser facilmente compreendidos mesmo por pessoas que não sabem programar.

Algorimtos podem ser testados mentalmente ou no papel, sem que tenham sido escritos em uma linguagem de programação. Esse tipo de teste pode ajudar a identificar erros antes de serem programados e até mesmo depois de programados, podem ajudar a identificar erros de lógica difícieis de serem compreendidos durante a execução do programa.

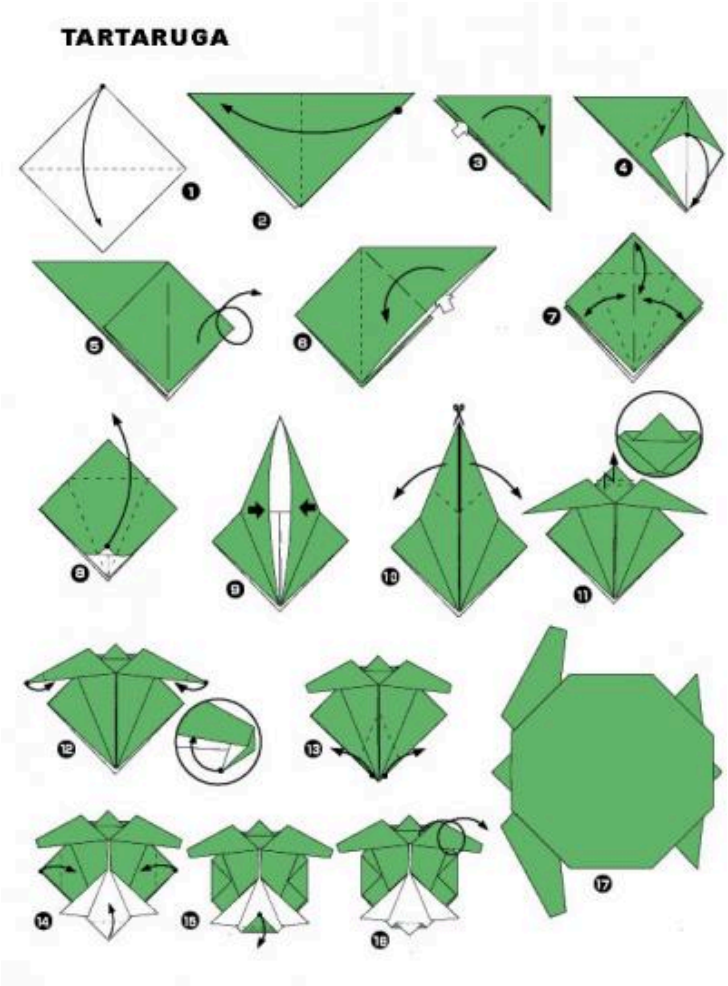
2.1. Tipos de algoritmos

Os algoritmos podem ser computacionais e não computacionais.

Vejamos a seguir exemplos desses dois casos.

2.2. Não computacionais

Algoritmo para montar um Origami



Algoritmo para trocar de uma lâmpada

Troca de lâmpada de rosquear - Manutenção | C&C Casa e Construção

1 - Coloque uma escada embaixo da lâmpada queimada.

2 - Escolha uma lâmpada nova de mesma potência/voltagem da queimada.

3 - Suba na escada até alcançar a lâmpada queimada.

4 - Gire a lâmpada queimada no sentido anti-horário até que ela se solte.

5 - Posicione a lâmpada nova no soquete.

6 - Gire a lâmpada no sentido horário, até que ela se firme.

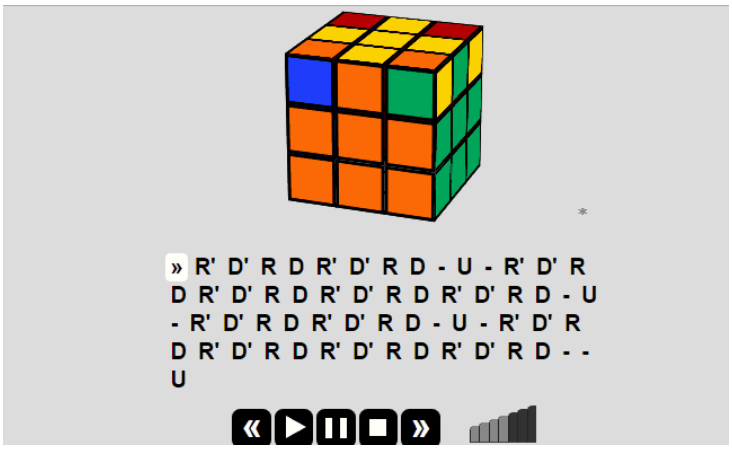
7 - Desça da escada.

8 - Guarde a escada.

9 - Descarte a lâmpada com segurança.

Outros exemplos:

- **Algoritmo para resolver um cubo mágico 3x3x3;**



Descontraindo ...

- **Algoritmo para sair de um ponto A e chegar a um ponto B.**

Observação:

Em termos gerais, um **manual de instruções para um jogo** não é considerado um algoritmo por si só.

Um manual de instruções geralmente contém uma sequência de passos e regras que descrevem como jogar o jogo, mas não necessariamente segue uma estrutura lógica de algoritmo para resolver um problema.

No entanto, é possível que um manual de instruções contenha elementos que possam ser considerados como algoritmos.

Por exemplo, **se o manual descrever uma estratégia específica para ganhar o jogo, essa estratégia pode ser vista como um algoritmo para alcançar o objetivo de vencer**.

Nesse caso, a estratégia seria uma sequência ordenada de instruções lógicas para alcançar um resultado específico.

Portanto, embora um manual de instruções para um jogo possa conter elementos que se assemelham a um algoritmo, em sua totalidade, ele geralmente não é considerado um algoritmo, mas sim um conjunto de orientações e regras para jogar o jogo de forma adequada.

2.3. Computacionais

Os algoritmos também são usados na programação computacional e na internet, muito comuns no dia a dia quando utilizamos motores de busca, por exemplo o Google, e nas redes sociais.

Exemplos:

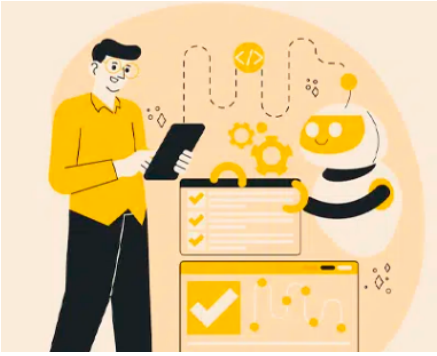
A) Algoritmos em motores de busca (Google)

Realiza diversas operações para organizar a maneira como o resultado das pesquisas aparecem para o usuário.

Quando um usuário pesquisa no Google por uma palavra-chave, como "o que é democracia", a ordem dos resultados mostrados depende de diversos cálculos que o algoritmo realizou para colocá-los naquela posição.

Esses cálculos estão baseados em diversas variáveis, como a qualidade e a atualidade do conteúdo, a qualidade do site onde está o conteúdo e quanto tempo as pessoas costumam ficar naquela página.

B) Algoritmos em redes sociais



As redes sociais também são programadas com complexos algoritmos. Os feeds do Facebook, Instagram e Twitter, por exemplo, mostram postagens e anúncios conforme o comportamento do usuário.

Isso significa que nem todas as postagens de seus amigos irão aparecer no seu feed. Os algoritmos dessas redes sociais selecionam as que fazem mais sentido conforme a sua preferência.

O algoritmo do Facebook, chamado de EdgeRank, por exemplo, seleciona os conteúdos mais relevantes para você de acordo com diversos fatores, como:

- Publicações e páginas que você curte;
- Pessoas com quem você interage;
- Páginas em que você deixa comentários;
- Seus compartilhamentos.

C) O Spotify

É outro exemplo de aplicativo e rede social que utiliza algoritmos para oferecer aos usuários uma experiência mais personalizada. Todas as semanas o aplicativo oferece uma playlist chamada “Descobertas da semana”.

Essa playlist contém 30 músicas que nunca foram tocadas pelo usuário no aplicativo, mas que são parecidas com outras músicas que ele costuma ouvir.

O objetivo é apresentar para o usuário as músicas e bandas que ele ainda não conhece, mas que provavelmente são compatíveis com o seu gosto musical.

Fonte: https://www.significados.com.br/algoritmo/

Outros exemplos

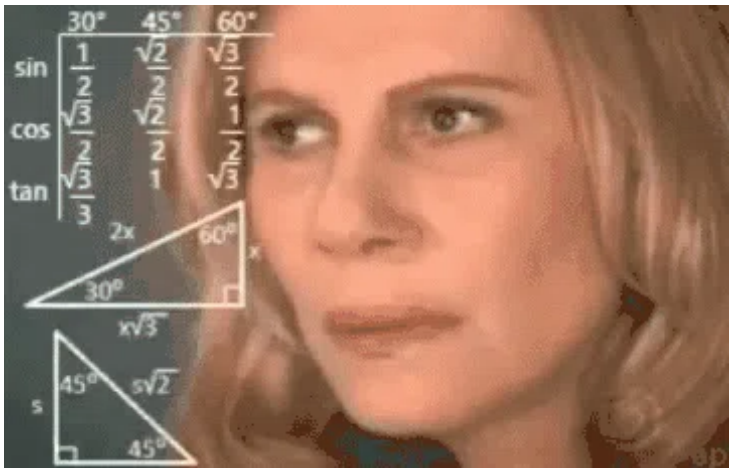
Aqui estão alguns dos principais tipos de algoritmos computacionais:

1. Algoritmos de **busca**: São usados para encontrar um elemento específico em uma coleção de dados, como um valor em uma lista ou um nó em uma árvore;
2. Algoritmos de **ordenação**: São utilizados para reorganizar os elementos de uma coleção de dados em uma determinada ordem, como ordenação em ordem crescente ou decrescente;
3. Algoritmos de **grafos**: São empregados para trabalhar com estruturas de dados de grafos, como encontrar o menor caminho entre dois nós, verificar a conectividade de um grafo, entre outros;
4. Algoritmos de **árvores**: São algoritmos que manipulam estruturas de dados de árvores, como percorrer uma árvore, inserir ou remover nós, encontrar o pai ou filhos de um nó, entre outras operações;
5. Algoritmos de **otimização**: São usados para encontrar a melhor solução ou o máximo ou mínimo global em um determinado problema, muitas vezes envolvendo algoritmos heurísticos ou meta-heurísticos;
6. Algoritmos de **aprendizado de máquina**: São algoritmos que permitem que os sistemas aprendam a partir de dados, construindo modelos e realizando previsões ou classificações com base nesses modelos;
7. Algoritmos de **criptografia**: São algoritmos utilizados para proteger informações e garantir a segurança da comunicação, como algoritmos de criptografia simétrica e assimétrica;
8. Algoritmos de **compressão**: São utilizados para reduzir o tamanho de dados ou arquivos, eliminando redundâncias e otimizando a representação dos dados;

entre outros.

3. Projeto de algoritmos

O conceito



Projetar é um processo que pressupõe a aplicação de várias técnicas e princípios com o propósito de definir um dispositivo, processo ou sistema em detalhes suficientes para permitir a sua construção física.

O principal objetivo ao projetar não é simplesmente produzir uma solução para o problema, mas sim é encontrar a melhor solução que satisfaça as limitações impostas pelos requisitos e pelos ambientes físico e social onde o sistema será usado.

Projeto de algoritmos

Ao projetar algoritmos, o objetivo é desenvolver uma sequência de etapas bem definidas e estruturadas que possam ser seguidas por um computador ou por um ser humano para resolver um problema.

Durante o processo de projetar algoritmos, é necessário compreender completamente o problema a ser resolvido e identificar as entradas necessárias, os passos de processamento e as saídas esperadas.

As formas de representação de algoritmos são utilizadas em diferentes etapas de desenvolvimento de software, nas fases de Análise e Projeto de software.

As principais formas de representação são:

- Linguagem natural;
- Fluxogramas;
- Pseudocódigo;
- Linguagem de programação.

3.1. Linguagem natural

Linguagem natural refere-se à forma de comunicação utilizada pelos seres humanos em sua interação cotidiana. É a linguagem que usamos para conversar, ler, escrever e transmitir informações de maneira verbal ou escrita. É uma forma de expressão que evoluiu naturalmente ao longo do tempo, desenvolvida e compartilhada entre os falantes de uma determinada comunidade ou cultura.

A linguagem natural é utilizada usualmente em algoritmos computacionais e não computacionais.

Em situações mais simples, a especificação por meio de texto é mais utilizada. Em situações mais complexas, os seres humanos tendem a utilizar notações mais elaboradas, tais como fluxogramas, notações matemáticas, etc.

Um exemplo de algoritmo não computacional elaborado em linguagem natural é a troca de uma lâmpada.

A) Troca de uma lâmpada

1 - Coloque uma escada embaixo da lâmpada queimada.

2 - Escolha uma lâmpada nova de mesma potência/voltagem da queimada.

3 - Suba na escada até alcançar a lâmpada queimada.

4 - Gire a lâmpada queimada no sentido anti-horário até que ela se solte.

5 - Posicione a lâmpada nova no soquete.

6 - Gire a lâmpada no sentido horário, até que ela se firme.

7 - Desça da escada.

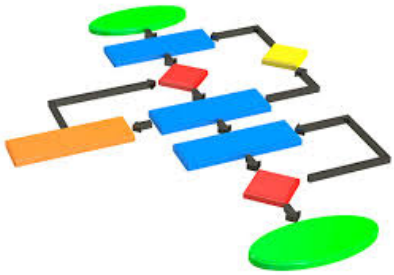
8 - Guarde a escada.

9 - Descarte a lâmpada com segurança.

3.2. Fluxogramas

O conceito

Um fluxograma é um tipo de diagrama. Trata-se de uma representação gráfica de um processo ou algoritmo. Os gráficos ilustram a execução de um processo.



Os fluxogramas são utilizados em projetos de software para representar a lógica interna ou funcionamento dos programas. Podem também ser usados para desenhar processos de negócio e o workflow (fluxo de trabalho) que envolve diversos atores corporativos no exercício de suas atribuições.

Os fluxogramas costumam ser utilizados para representar casos complexos, e principalmente, processos de negócio.

Diagramas de blocos

Um diagrama de blocos, também conhecido como diagrama de blocos funcionais, é uma representação visual de um sistema, processo ou algoritmo por meio de blocos interconectados.

Cada bloco representa uma função ou operação específica e a conexão entre os blocos indica a sequência ou o fluxo de dados entre eles.

Quadro: Diagramas de blocos para algoritmos

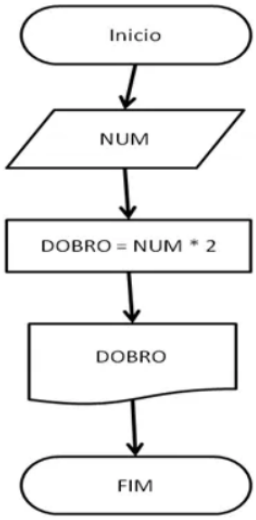
Nomenclatura	Símbolo	Função
Terminal		Indica início ou fim de um algoritmo.
Entrada		Indica entrada de dados manual.
Saída		Indica saída de dados na tela.
Processamento		Operação de atribuição.
Decisão		Indica quando há alguma decisão a ser tomada.
Repetição		Indica o limite de um laço de repetição.

Os diagramas de blocos são amplamente utilizados em diversas áreas, incluindo engenharia, ciência da computação, controle de processos, eletrônica e muitas outras disciplinas. Eles permitem uma representação visual clara e simplificada de sistemas complexos, auxiliando na compreensão, comunicação e análise.

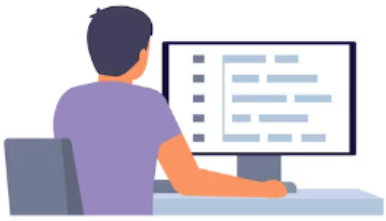
Cada bloco em um diagrama de blocos representa uma função ou operação que pode ser executada, como cálculos matemáticos, tomadas de decisão, entradas e saídas de dados, entre outros. Os blocos são interconectados por setas que indicam a direção do fluxo de dados ou controle.

Os diagramas de blocos podem variar em complexidade, desde diagramas simples com alguns blocos interconectados até diagramas mais elaborados com vários níveis de detalhamento. Eles são úteis para representar algoritmos, fluxos de processos, arquiteturas de sistemas, circuitos eletrônicos e muito mais.

Exemplo de um fluxograma para calcular e mostrar o dobro de um número informado pelo usuário.



3.3. Pseudocódigo



O conceito

Pseudocódigo é uma forma genérica de escrever um algoritmo, utilizando uma linguagem simples (nativa a quem o escreve, de forma a ser entendida por qualquer pessoa) sem necessidade de conhecer nenhuma linguagem de programação.

Aqui no Brasil, utilizamos a notação de pseudocódigo em língua portuguesa, que ficou também conhecida como **PORTUGOL**.

- O Portugol não é uma Linguagem de programação, mas sim uma pseudo (falsa) linguagem ou um método.
- Foi criado pelos professores Antonio Carlos Nicolodi (Brasil) e Antonio Mannso (Portugal) em 1986.
- Originalmente iniciou-se como uma tradução da linguagem de programação Pascal para a língua portuguesa.
- O seu nome é uma mistura das três palavras: Português, das linguagens de programação Algol e Pascal, (PORTUguês, ALGOL e PascaL).

Exemplo de um algoritmo escrito pseudo código em Portugol:

O algorimto a seguir foi escrito para:
1. solicitar para o usuário digitar um valore;
2. calcular o dobro do valor
3. mostrar o resultado do cálculo ao usuário

```
escreva ("Informe o valor para cálculo de seu dobro")
leia (valor)
dobro <= valor * 2
escreva ("O valor do dobro é ", dobro)
```

3.4. Para saber mais: comandos em pseudocódigo

Lista dos principais comandos em pseudocódigo:

Estrutura sequencial

a) Entrada de dados:
Exemplo: ler ou leia (valor)

b) Atribuição de valores:
Exemplo: variavel <- valor

c) Saída de dados:
Exemplo: escrever ou escreva (valor)

Estruturas condicionais

a) se-então-senão:

```
se (condicao) então
   // código a ser executado se a condição for verdadeira
senão
   // código a ser executado se a condição for falsa
fim_se
```

Estruturas de repetição

a) para:

```
para i de valor_inicial até valor_final faça
   // código a ser executado repetidamente com base em um intervalo
fim_para
```

b) enquanto:

```
enquanto (condicao) faça
   // código a ser repetido enquanto a condição for verdadeira
fim_enquanto
```

c) repita:

```
contador ← 1
Repita
   Escreva contador
   contador ← contador + 1
Até contador > 5
```

3.5. Linguagens de Programação

Linguanges de programação tem diferentes formas de representação de comandos algoritmicos. Algumas são semelhantes, outras bem distintas.

Exemplos de representações de algoritmo em algumas linguagens de programação.

a) Linguagem natural

Algoritmo: Calcule o dobro
O algorimto a seguir foi escrito para:
1. solicitar para o usuário digitar um valore;
2. calcular o dobro do valor
3. mostrar o resultado do cálculo ao usuário

b) Pseudo-código

```
escreva ("Informe o valor para cálculo de seu dobro")
leia (valor)
dobro <= valor * 2
escreva ("O valor do dobro é ", dobro)
```

---

c) Linguagens de programação

Representação em Basic

```
PRINT "Informe o valor para cálculo de seu dobro"
INPUT valor
dobro = valor * 2
PRINT "O valor do dobro é "; dobro
```

Representação em Pascal

```
program CalculoDobro;
var
  valor, dobro: Integer;
begin
  Write('Informe o valor para cálculo de seu dobro: ');
  ReadLn(valor);
  dobro := valor * 2;
  WriteLn('O valor do dobro é ', dobro);
end.
```

Representação em C

```
#include <stdio.h>
int main() {
    int valor, dobro;
    printf("Informe o valor para cálculo de seu dobro: ");
    scanf("%d", &valor);
    dobro = valor * 2;
    printf("O valor do dobro é %d\n", dobro);
    return 0;
}
```

Representação em Python

```
valor = int(input("Informe o valor para cálculo de seu dobro: "))
dobro = valor * 2
print("O valor do dobro é", dobro)
```

4. Estrutura fundamental de um algoritmo

A estrutura fundamental de um algorimto é dividida nas seguintes seções:

- Cabeçalho;
- Área de declaração de variáveis;
- Programa.

Veja a seguir:

Exemplo:

Seções do algoritmo ou programa	**Exemplo**
Cabeçalho	# Programa: Calculadora de Quadrado # Autor: [Seu nome] # Data: 9 de maio de 2024 # Versão: 1.0
Declaração de variáveis	#número, do tipo decimal (ponto flutuante)
Programa	#entrada de dados numero = float(input("Digite um número: ")) #processo de cálculo quadrado = numero ** 2 • #saída • print (“O quadrado do número ”, numero, “ é ”, quadrado)

4.1. Cabeçalho

O **cabeçalho** de um programa é uma parte específica localizada no início do código-fonte que geralmente contém informações importantes sobre o programa.

Essas informações são destinadas a fornecer detalhes sobre o propósito do programa, o autor, a data de criação, a versão, as bibliotecas utilizadas, entre outros detalhes relevantes.

Em muitas linguagens de programação, incluindo Python, o cabeçalho é colocado como um comentário especial no início do arquivo-fonte. Em Python, um comentário começa com o caractere "#" (hash) e é ignorado pelo interpretador durante a execução.

O cabeçalho é uma forma de documentar o código para que outras pessoas (ou você mesmo, no futuro) possam entender o propósito e os detalhes do programa sem a necessidade de analisar todo o código.

Exemplo de cabeçalho em python para o algoritmo do dobro.

```
# Programa: CalculoDobro
# Descrição: Este programa solicita um valor ao usuário e calcula o dobro desse valor.
# Autor: [Seu Nome]
# Data de Criação: 03/11/2024
# Versão: 1.0
# Bibliotecas Utilizadas: Nenhuma
# Este programa foi desenvolvido como exemplo de implementação em Python.
# O objetivo é demonstrar o uso de entrada e saída de dados, além de operações aritméticas básicas.
```

4.2. Área de declaração de variáveis

Nesta parte, você pode declarar e documentar as variáveis que serão usadas ao longo do programa.

- As **variáveis** são usadas para armazenar dados e podem ser modificadas durante a execução do código.
- As **constantes** são variáveis cujo valor não será alterado durante a execução do programa e, geralmente, são escritas em letras maiúsculas.

Em **Python**, não é necessário declarar o tipo das variáveis explicitamente, mas podemos documentá-las para fins de verificação de conteúdos em futuras manutenções do código:

```
# Declaração de variáveis
valor    # variável que armazenará o valor informado pelo usuário
dobro    # variável que armazenará o resultado do dobro
```

4.3. Programa principal e subprogramas

Programa principal e subprogramas

Aqui é onde a lógica principal do programa está localizada. As instruções são executadas a princípio em ordem sequencial mas podem podem incluir outros tipos de estrutura de controle de fluxo de execução.

No programa principal e subprogramas tambem podemos criar documentações de separações de seções, como por exemplo, #entrada, #processo, #saída e outras anotações internas que se fizerem necessárias.

Exemplo completo para o algorimto do cálculo do dobro, com todos os elementos:

```
# Cabeçalho
# Programa: CalculoDobro
# Descrição: Este programa solicita um valor ao usuário e calcula o dobro desse valor.
# Autor: [Seu Nome]
# Data de Criação: 03/11/2024
# Versão: 1.0
# Bibliotecas Utilizadas: Nenhuma


# Área de declaração de variáveis
valor = 0    # variável que armazenará o valor informado pelo usuário
dobro = 0    # variável que armazenará o resultado do dobro
FACTOR = 2   # constante para multiplicação


# Programa principal
# Entrada
valor = int(input("Informe o valor para cálculo de seu dobro: "))
# Processo
dobro = valor * FACTOR
# Saída
print("O valor do dobro é", dobro)
```

4.4. Documentação interna

Documentação interna de um algoritmo

A documentação interna de um algoritmo é uma descrição detalhada do funcionamento interno do algoritmo, incluindo sua lógica, estrutura de dados utilizada, fluxo de controle e quaisquer outras informações relevantes para entender como o algoritmo realiza sua tarefa.

Essa documentação é voltada para os desenvolvedores ou pessoas que precisam entender e potencialmente modificar ou otimizar o algoritmo, na forma de comentários no decorrer do código.

Comentários

São linhas de textos que cuja marcação inicial e final indicam ao Python que o texto não contém comandos para serem processados.

Comandos para comentários em Python

Em Python, você pode usar o caractere `#` e as aspas triplas (`"""`) para documentar o código.

a) Comentários com `#`:

- Os comentários em linha começam com o caractere `#` e são usados para fornecer explicações breves e diretas sobre partes específicas do código.
- **A cerquilha serve para acrescentar apenas uma linha de comentário ou isolar o funcionamento dela.**
- Eles são úteis para explicar o propósito ou a lógica por trás de uma linha de código ou de um pequeno bloco de código.
- Os comentários com `#` não são interpretados pelo interpretador Python e são ignorados durante a execução do programa.
- Eles são úteis para fornecer insights rápidos para quem lê o código.

Exemplo:

print ("Olá mundo!") #esse comando mostra a mensagem "Olá mundo!" na tela do computador.

b) Comentários aspas triplas (`"""`)

- As aspas triplas são utilizadas para acrescentar várias linhas de código, demarcando o início e o final delas.
- Aspas triplas também servem para isolar o funcionamento de trechos específicos de código.

Exemplo:

"""
#esse trecho do programa calcula ...
aceita várias linhas, veja que não é necessário utilizar a cerquilha
"""

4.5. Indentação

Em ciência da computação, indentação é um termo aplicado ao código fonte de um programa para ressaltar ou definir a estrutura do algoritmo.

Na maioria das linguagens de programação, a indentação é empregada com o objetivo de ressaltar a estrutura do algoritmo, aumentando assim a legibilidade do código.

A indentação é uma característica importante no Python, pois além de promover a legibilidade é essencial para o bom funcionamento do código. Em outras palavras, se a indentação não estiver adequada o programa pode se comportar de forma inesperada ou até mesmo não compilar.

Na estrutura sequencial não há indentação. Isso significa que não é permitido deslocar linhas à direita além da posição zero na linha. Caso contrário o programa não irá executar e retornará mensagem de erro.

Veja a seguir:

Figura: Algoritmo indentado corretamente

```
main.py
1 print ("digite uma palavra")
2 palavra = input ()
3 print ("a palavra digitada foi ", palavra)
4

digite uma palavra
oi
a palavra digitada foi  oi


...Program finished with exit code 0
Press ENTER to exit console.
```

Veja a seguir a interferência da indentação ou deslocamento da segunda linha do código no programa executado.

Figura: Algoritmo com indentação incorreta

```
main.py
1 print ("digite uma palavra")
2     palavra = input ()
3 print ("a palavra digitada foi ", palavra)
4

  File "/home/main.py", line 2
    palavra = input ()
IndentationError: unexpected indent


...Program finished with exit code 1
Press ENTER to exit console.
```

Veja que a mensagem de erro foi: IndentationErro: unexpected indent

Tradução: Erro de indentação: indentação inesperada

5. Estruturas de controle

Estrutura de controle (ou fluxo de controle) refere-se à ordem em que instruções ou comandos são executados.

Os tipos de estruturas de controle disponíveis diferem de linguagem para linguagem, mas podem ser caracterizados por seus efeitos.

- O primeiro é a **sequencialidade** da execução das instruções, chamada estrutura sequencial.
- O segundo é a execução de um bloco de código somente se uma **condição** é verdadeira, uma estrutura de seleção.
- O terceiro é a execução de um bloco de código de forma a **repetir** uma coleção de instruções ou comandos.
- O quarto é a execução de instruções distantes entre si, como blocos ou funções isoladas, em que o controle de fluxo possivelmente volte para a posição original posteriormente, como chamadas de **subrotinas**.
- O quinto é a **parada** (pausa) ou **finalização** do programa de computador.

6. Estrutura Sequencial

Em ciência da computação, uma estrutura sequencial é uma estrutura de orientação do fluxo de execução dos instruções de programação uma linguagens de programação.

A estrutura de orientação determina que a execução dos comandos seja sequencial:

- iniciando com o primeiro comando ou primeira linha do editor de código
- depois a segunda linha
- depois a terceira linha
- e assim sucessivamente, sem desvios,

uma seguida da outra, na ordem em que foram declarados no código fonte.

Como exemplo, o código a seguir vai ser executado na ordem em que aparece, de cima para baixo:

Pseudo-código

```
escreva ("Informe o valor para cálculo de seu dobro")
leia (valor)
dobro <= valor * 2
escreva ("O valor do dobro é ", dobro)
```

6.1. Organização lógica da estrutura sequencial

A organização lógica da estrutura sequencial, na grande maioria dos problemas, pressupõe a seguinte forma:

- **Entrada -> processo -> saída.**

Exemplos

A seguir, apresento a representação do algoritmo do cálculo do dobro nas 3 representações: linguagem natural, pseudo-código e Python.

Especificação do problema em linguagem natural

O algorimto a seguir foi escrito para:
1. solicitar para o usuário digitar um valor;
2. calcular o dobro do valor
3. mostrar o resultado do cálculo ao usuário

Quadro: Entrada, processo e saída e suas representações

Seções principais	**Linguagem Natural**	**Pseudo-código**	**Python**
Entrada	Exiba uma mensagem solicitando ao usuário que informe um valor para o cálculo do dobro. Receba esse valor e armazene-o na variável valor	// Entrada MOSTRAR "Informe o valor para cálculo de seu dobro:" LER valor	# Entrada valor = int(input("Informe o valor para cálculo de seu dobro: "))
Processo	Calcule o dobro do valor informado, multiplicando o valor armazenado em valor por 2. Armazene o resultado na variável dobro.	// Processo dobro <- valor * 2	# Processo dobro = valor * 2
Saída	Exiba uma mensagem para o usuário com o resultado, indicando o valor do dobro.	// Saída MOSTRAR "O valor do dobro é ", dobro	# Saída print ("O valor do dobro é", dobro)

7. Categorias de problemas

O uso da estrutura sequencial em programação de computadores podem abranger várias categorias de aplicação direta de fórmulas matemáticas e cálculos simples.

Algumas das categorias mais comuns incluem:

Entrada e Saída de Dados: Problemas que envolvem a leitura e escrita de dados em tela ou arquivos, bem como o uso de formatação de saída.

Cálculos Matemáticos básicos : Problemas que envolvem operações matemáticas básicas, como adição, subtração, multiplicação e divisão, bem como cálculos de potência, raiz quadrada e outras operações aritméticas ou complexos.

Conversões de Idade, data, hora: Problemas que requerem o cálculo da idade de uma pessoa com base em sua data de nascimento ou cálculos de intervalo de tempo entre duas datas. Problemas relacionados à manipulação de datas e horas, como cálculo de dias entre datas, exibição de datas formatadas, etc.

Conversão de Unidades: Problemas que requerem a conversão de unidades de medida, como converter temperatura de Celsius para Fahrenheit, metros para centímetros, quilogramas para gramas, etc.

Fórmulas: Problemas que envolvem o uso de fórmulas em geral, fórmulas geométricas para calcular áreas, perímetros, volumes, circunferências, etc.

Médias e Estatísticas: Problemas que envolvem o cálculo de médias aritméticas, médias ponderadas, variância, desvio padrão e outros conceitos estatísticos simples.

Porcentagem e Financeiros: Problemas que exigem o cálculo de porcentagem, como calcular descontos, acréscimos, juros e taxas percentuais. Problemas relacionados a cálculos financeiros, como cálculo de salários líquidos, descontos de INSS e IRPF, cálculo de prestações, entre outros.

Manipulação de Strings: Problemas que envolvem operações básicas com strings, como concatenação, contagem de caracteres, extração de substrings, etc.

8. Verificação e validação de algoritmos

Quando elaboramos um fluxograma ou um algoritmo, podemos fazer verificação e validação.

Verificação

O processo de verificação é executado em 3 etapas:

1a.) Checagem

- se a ortografia dos comandos está correta;
- se os elementos de entrada de dados foram definidos corretamente (variáveis, tipos, leitura e atribuição);
- se os comandos e a ordem de execução do processo estão corretos;
- se os elementos de saída de dados foram definidos corretamente.

2a.) Planejamento de casos de teste

- uma vez que a checagem foi realizada com sucesso, deve-se planejar os casos de teste;
- os casos de teste mais comuns são baseados na funcionalidade.
  - Observa-se a etapa de entrada de dados, processo e saída de dados;
  - Prepara-se um conjunto de dados de entrada;
  - Efetuam-se cálculos para determinar os resultados de saída do processo;

3a.) Execução de Teste de mesa

- Utilizamos a abordagem de teste de mesa para simular o funcionamento de um algoritmo sem estar implementado em uma linguagem de programação.
- Aplicamos os casos de teste
  - Utilizamos os dados de entrada
  - Confrontamos o resultado do processamento mental com os dados esperados já calculados do teste de mesa.

Validação

O processo de validação é a **execução do caso de teste no programa** executado em um computador.

8.1. Teste de Mesa

Um **teste de mesa** é uma técnica utilizada para simular manualmente a execução de um algoritmo ou programa, linha por linha, com o objetivo de verificar seu funcionamento e identificar possíveis erros lógicos. Ele é especialmente útil para programadores, estudantes e profissionais que desejam entender o comportamento do código antes de sua execução em um computador.

Como Funciona o Teste de Mesa

No teste de mesa, é criado um quadro ou tabela onde são anotados os valores das variáveis e o fluxo de execução do algoritmo. Cada linha ou instrução do código é analisada e executada mentalmente, registrando:

1. **Valores das Variáveis**: Os valores são atualizados em cada linha onde ocorre alguma alteração.
2. **Fluxo de Controle**: As instruções de controle (como `if`, `while`, `for`) são seguidas para verificar o caminho que o algoritmo está percorrendo.
3. **Resultados Parciais**: Valores intermediários, saídas temporárias e outras informações são anotadas para verificar o comportamento do algoritmo em diferentes etapas.

o teste de mesa, cada linha do código é analisada e executada manualmente, e os valores das variáveis são registrados, linha a linha. Abaixo está uma explicação detalhada dos campos da tabela e como eles são preenchidos em um caso de teste.

Estrutura da Tabela de Caso de Teste

1. **Código**:
   - Nesta coluna, são listadas as instruções ou linhas do código que estão sendo executadas. Cada linha representa um passo específico do algoritmo. A ideia é descrever cada linha para entender como o valor das variáveis muda ao longo da execução.
2. **Variáveis e Constantes**:
   - Cada coluna representa uma variável ou constante usada no código. Aqui, são registradas as atualizações e valores atuais de cada variável após a execução da linha correspondente do código. Isso permite acompanhar as mudanças em todas as variáveis durante o fluxo de execução do algoritmo.
3. **Saída de Dados na Tela**:
   - Esta coluna representa a saída do programa, simulando o que seria exibido ao usuário. Cada linha mostra a saída gerada após a execução da instrução correspondente, permitindo visualizar o que o usuário veria na tela em cada etapa.

8.2. Exemplo de teste de mesa

Vamos usar a estrutura para exemplificar com um caso de teste do cálculo do dobro de um número:

Algoritmo

```
valor = int(input("Informe um valor: "))  # Linha 1
dobro = valor * 2                         # Linha 2
print("O valor do dobro é", dobro)         # Linha 3
```

Tabela de Caso de Teste

Código	valor	dobro	Saída de dados na tela
`valor = int(input("Informe um valor: "))`	5	-	Informe um valor:
`dobro = valor * 2`	5	10	-
`print("O valor do dobro é", dobro)`	5	10	O valor do dobro é 10

Explicação do Exemplo

1. **Linha 1 do Código: Solicita ao usuário para inserir um valor e armazena-o na variável valor.**
   - valor recebe o valor 5 (valor de entrada do usuário).
   - dobro ainda não possui valor definido.
   - A saída na tela mostra: "Informe um valor: ".
2. **Linha 2 do Código: Calcula o dobro do valor e armazena o resultado na variável dobro.**
   - valor mantém o valor 5.
   - dobro é atualizado para 10.
   - Não há saída na tela para esta linha.
3. **Linha 3 do Código: Exibe o valor do dobro calculado.**
   - valor e dobro mantêm seus valores (5 e 10, respectivamente).
   - A saída na tela é: "O valor do dobro é 10".

8.3. Como elaborar caso de teste

1. **Definir o Objetivo do Caso de Teste**
   O objetivo deste teste é verificar se o algoritmo para calcular o dobro de um número funciona corretamente ao receber a entrada mínima válida, que é 1. Queremos assegurar que o algoritmo realiza o cálculo e exibe o resultado esperado.

2. **Identificar as Variáveis e Constantes**
   Determine as variáveis e constantes que serão acompanhadas na tabela:

   - valor: variável que armazena o valor informado pelo usuário.
   - dobro: variável que armazena o resultado do cálculo do dobro.

3. **Selecionar o Valor de Teste**
   Escolha o valor 1 como a entrada mínima válida para valor. Esse valor foi escolhido porque é o menor número positivo que mantém o caso válido e simples.

4. **Estabelecer o Resultado Esperado**
   Calcule manualmente o resultado esperado:

   - Se valor = 1, então dobro = valor * 2 = 1 * 2 = 2.

5. **Preencher a Estrutura do Caso de Teste em Tabela**
   Utilizando a estrutura da tabela fornecida, planeje o caso de teste registrando cada passo do algoritmo e as mudanças nas variáveis.

Estrutura do Caso de Teste em Tabela

Código	**valor**	**dobro**	**Saída de dados na tela**
valor = int(input("Informe um valor: "))	1	-	Informe um valor:
dobro = valor * 2	1	2	-
print("O valor do dobro é", dobro)	1	2	O valor do dobro é 2

Explicação do Caso de Teste

1. **Linha 1 do Código:**

   - Solicita ao usuário para inserir um valor e armazena-o na variável valor.
   - Valor das Variáveis: valor recebe 1, dobro ainda não possui valor.
   - Saída na Tela: "Informe um valor: " (solicitação de entrada).

2. **Linha 2 do Código:**

   - Calcula o dobro do valor e armazena o resultado em dobro.
   - Valor das Variáveis: valor mantém 1, dobro é atualizado para 2.
   - Saída na Tela: Não há saída para esta linha.

3. **Linha 3 do Código:**

   - Exibe o valor do dobro calculado.
   - Valor das Variáveis: valor permanece 1, dobro permanece 2.
   - Saída na Tela: "O valor do dobro é 2".

Revisão e Conclusão

Revise a tabela para assegurar que todos os passos foram corretamente seguidos e que o resultado obtido está de acordo com o esperado. Caso o resultado obtido seja o mesmo que o esperado, o teste é considerado Aprovado.